



LEARNING HANDOUT

PHP Forms Processing and State Management

Course	Week	Term	School Year
IT WST 21 – Web Systems and Technologies 2	Week 7 (Midterm Period)	1st Semester	2026–2027

Learning Outcome

By the end of this lesson, you should be able to develop database-driven web applications using PHP and MySQL integration techniques.

So far you have written PHP scripts that run entirely on their own. Real web applications need to talk to the user — receiving form input, remembering who is logged in, and handling anything that goes wrong along the way. This week introduces the tools PHP gives you to do exactly that.

1. GET and POST Methods

HTML forms send data to the server using an HTTP method. PHP automatically collects this data into two superglobal arrays: \$_GET and \$_POST.

Aspect	GET	POST
Where data appears	Visible in the URL (e.g., ?name=Juan)	Hidden in the request body
Typical use	Search boxes, filters, pagination links	Login forms, registrations, sensitive data
Data limit	Limited by URL length	Much larger data allowed
Bookmarkable / cacheable	Yes	No

A Simple HTML Form

```
<form action="process.php" method="POST">
  Name: <input type="text" name="studentName">
  <input type="submit" value="Submit">
</form>
```

Reading Submitted Data in PHP

```
<?php
// process.php
```

```
$name = $_POST["studentName"];
echo "Hello, " . $name . "!";
?>
```

- \$_GET reads values passed in the URL query string; \$_POST reads values sent in the form body.
- \$_REQUEST can read either one, but explicitly using \$_GET or \$_POST is clearer and safer.

2. Form Handling and Validation

Never trust data coming from a form. Validation checks that the submitted data is present, correctly formatted, and safe before your script uses it.

Checking Whether a Form Was Submitted

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // form was submitted, process it
}
?>
```

Basic Validation Example

```
<?php
$name = trim($_POST["studentName"] ?? "");
$email = trim($_POST["email"] ?? "");
$errors = [];

if ($name === "") {
    $errors[] = "Name is required.";
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Please enter a valid email address.";
}

if (empty($errors)) {
    echo "Form submitted successfully!";
} else {
    foreach ($errors as $error) {
        echo $error . "<br>";
    }
}
?>
```

- trim() removes extra whitespace; the ?? "" default avoids an error if the field is missing.
- filter_var() with FILTER_VALIDATE_EMAIL is a built-in way to check email format.
- htmlspecialchars() should be used when redisplaying user input, to prevent malicious HTML/JavaScript from running.

3. Sessions and Cookies

HTTP is stateless — by default, the server does not remember anything between page loads. Sessions and cookies solve this by storing data related to a specific visitor.

Sessions

A session stores data on the server, tied to a unique session ID that PHP keeps track of in the visitor's browser. Sessions are ideal for data like login status.

```
<?php
    session_start();          // must be called before any output
    $_SESSION["username"] = "jdelacruz";

    echo "Welcome back, " . $_SESSION["username"];
?>

<?php
    session_start();
    session_destroy();      // ends the session (e.g., on logout)
?>
```

Cookies

A cookie stores small pieces of data directly in the visitor's browser, which is sent back to the server on every request. Cookies are ideal for preferences that should persist across visits.

```
<?php
    // set a cookie
    that lasts 30 days

    setcookie("theme",
    "dark", time() +
    (30 * 24 * 60 *
    60));

    // read it on a
    later request
    echo
    $_COOKIE["theme"]
    ?? "light";
?>
```

Aspect	Session	Cookie
Stored on	Server (browser only keeps the session ID)	Client's browser
Typical use	Login state, cart contents during checkout	Remembered preferences, "remember me"
Lifespan	Ends when the session/browser closes (by default)	Set an explicit expiration time

4. Error Handling Techniques

Well-built applications anticipate that things can go wrong — a missing field, a failed database connection, an invalid file. PHP gives you tools to handle these situations gracefully instead of letting the script crash.

try / catch / finally

```
<?php
```

```
try {
    $result = 10 / $_POST["divisor"];
    echo "Result: " . $result;
} catch (DivisionByZeroError $e) {
    echo "Error: cannot divide by zero.";
} finally {
    echo "<br>Calculation attempt finished.";
}
?>
```

Throwing Custom Exceptions

```
<?php
function connectToDatabase($host) {
    if (empty($host)) {
        throw new Exception("Database host cannot be empty.");
    }
    // connection logic here
}

try {
    connectToDatabase("");
} catch (Exception $e) {
    echo "Caught error: " . $e->getMessage();
}
?>
```

- try holds the code that might fail; catch handles the specific error type; finally always runs, whether or not an error occurred.
- throw new Exception(...) lets you raise your own errors with a custom, meaningful message.
- Good error handling protects your application from crashing and gives the user a clear, friendly message instead of a raw PHP error.

5. Key Takeaways

- \$_GET and \$_POST let a PHP script receive data submitted through an HTML form.
- Always validate and sanitize form input before using or storing it — never trust data from the user.
- Sessions store data on the server across page loads (great for login state); cookies store data in the browser (great for preferences).
- try / catch / finally and thrown exceptions let your script handle errors gracefully instead of crashing.
- Together, these tools let PHP build stateful, interactive, database-driven web applications.

6. Learning Resources / References

PHP Official Documentation — <https://www.php.net/docs.php>

PHP Manual — <https://www.php.net/manual/en/>

W3Schools PHP Tutorial — <https://www.w3schools.com/php/>

Nixon, R. (2021). Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5 (6th ed.). O'Reilly Media.

7. Practice Exercises

Complete the following exercises to prepare for our demonstration and hands-on laboratory activities:

1. Create a registration form (name, email, password) that submits via POST, and a process.php script that displays each submitted value.
2. Add validation to the form above: all fields required, and the email must pass FILTER_VALIDATE_EMAIL.
3. Build a simple login page that stores the username in \$_SESSION after a successful login, and a logout.php that destroys the session.
4. Use setcookie() to remember a user's preferred theme (light/dark) for 7 days, and display the current theme on page load.
5. Wrap a division operation in try / catch / finally so that dividing by zero shows a friendly error message instead of crashing the script.